

RACE Reading Comprehension Project Report

Abstract

This project builds an end-to-end reading-comprehension quiz system on the RACE dataset. The system preprocesses official train/dev/test splits, validates data quality and leakage, and converts each multiple-choice question into model-ready candidate-option rows. Model A performs answer verification with a rubric-aligned classical LR + XGBoost ensemble using sparse lexical features, cosine/overlap features, and within-question relative features; it also includes a supervised template-based question generator and an optional BERT multiple-choice backend. Model B generates plausible distractors and progressive hints using passage candidate extraction, answer-type controls, supervised ranking, and evidence-sentence selection. Evaluation uses BLEU, ROUGE-L, METEOR, exact-match diagnostics, Precision@3, Recall@3, F1@3, top-1 hint accuracy, and MRR. The best classical Model A reaches test BLEU 0.4106, ROUGE-L 0.4922, METEOR 0.4438, and exact 0.3739, while the best BERT multiple-choice checkpoint reaches test BLEU 0.5431 and exact 0.5172. The Streamlit UI combines question generation, answer checking, distractors, hints, analytics, and backend comparison for demonstration.

Introduction & Motivation

Reading-comprehension systems are useful when they can do more than select an answer: they should also explain, generate practice material, and support an interactive learner workflow. This project addresses that broader goal by combining answer verification, question generation, distractor generation, hint generation, and a Streamlit quiz interface. The RACE dataset is a strong fit because it contains middle-school and high-school exam passages with four-option multiple-choice questions that require both lexical matching and reasoning [1]. The motivation is to build a reproducible university-project pipeline that satisfies traditional machine-learning requirements while also comparing an advanced neural BERT backend.

Related Work

The dataset foundation follows RACE, a large-scale reading-comprehension benchmark collected from English examinations and designed to test understanding and reasoning [1]. Logistic Regression provides a traditional supervised baseline for binary option scoring [2], while Random Forests and XGBoost support non-linear tree-based ranking and boosted decision-tree comparison [3, 4]. The optional neural backend uses BERT, which fine-tunes bidirectional Transformer representations for downstream NLP tasks including question answering [5]. Finally, BLEU, ROUGE, and METEOR are established automatic text-overlap metrics that support the answer-text and distractor-text evaluation discussion [6, 7, 8].

Project Context

This project uses the local RACE dataset for an answer-verification style multiple-choice reading-comprehension task. Each question has four candidate answers. The model ranks the answer options,

converts the selected option letter back to answer text, and evaluates that text with BLEU, ROUGE-L, and METEOR.

Why EDA Was Needed

- It confirms the provided local RACE folder was loaded correctly and that train, dev, and test splits are present.
- It checks data quality before training: missing values, invalid labels, duplicate question ids, and split leakage.
- It studies answer-position balance, text length, middle-vs-high differences, and possible dataset artifacts.
- It creates baseline scores so the trained model can be compared with simple non-ML approaches.

Preprocessing Applied

1. Local dataset ingestion: JSON text files are read from RACE/train, RACE/dev, and RACE/test, including both middle and high folders.
2. Standard schema: each question becomes one CSV row with example_id, question_id, split, level, article, question, answer, correct_answer_text, and options A to D.
3. Text normalization: whitespace, newlines, non-breaking spaces, curly quotes, and dash variants are normalized without removing meaningful words or negations.
4. Validation rules: required fields are checked for missing or empty values, answers must be one of A/B/C/D, and question_id must be unique inside each split.
5. Leakage control: official train/dev/test folders are preserved, and exact example_id plus article overlaps are checked across splits.
6. Model-ready reshaping: each wide MCQ row is converted into four candidate-option rows with a binary label for answer verification.
7. Train-only feature fitting: CountVectorizer and TF-IDF are fitted only on training text and then reused for dev/test transformation.
8. Evaluation preparation: predicted letters are mapped back to predicted answer text before BLEU, ROUGE-L, and METEOR are calculated.

Dataset Analysis

The dataset analysis checks split size, school level distribution, text length, answer-position balance, leakage, lexical overlap, option-length bias, and simple baselines before model training. These checks establish that the train/dev/test split is usable and that lexical overlap is helpful but not sufficient as a final decision rule.

Dataset Overview

split	questions	passages	avg_article_w ords	avg_question _words	avg_correct_a nswer_words
dev	4887	1389	275.3000	8.9443	5.9515
test	4934	1407	277.0103	9.0529	6.0614
train	87866	25135	278.6729	9.0999	5.9533

By split and level:

split	level	questions	passages	avg_questions_per_passage	avg_article_words	avg_question_words	avg_correct_answer_words
dev	high	3451	1021	3.3800	308.9646	9.3387	6.4512
dev	middle	1436	368	3.9022	194.3969	7.9965	4.7507
test	high	3498	1045	3.3474	309.7521	9.4891	6.5835
test	middle	1436	362	3.9669	197.2535	7.9903	4.7897
train	high	62445	18726	3.3347	313.2691	9.5083	6.4704
train	middle	25421	6409	3.9665	193.6895	8.0967	4.6831

Data Quality Findings

All preprocessing validation checks passed: no missing required text, no invalid answer labels, and no duplicate question ids were found.

Missing/empty field summary:

split	column	missing_or_empty
train	article	0
train	question	0
train	answer	0
train	A	0
train	B	0
train	C	0
train	D	0
train	correct_answer_text	0
dev	article	0
dev	question	0
dev	answer	0
dev	A	0
dev	B	0
dev	C	0
dev	D	0
dev	correct_answer_text	0
test	article	0
test	question	0
test	answer	0
test	A	0
test	B	0
test	C	0
test	D	0
test	correct_answer_text	0

Answer Distribution Findings

split	answer	count	percentage
dev	A	1067	21.8334
dev	B	1291	26.4170
dev	C	1303	26.6626
dev	D	1226	25.0870
test	A	1063	21.5444
test	B	1323	26.8139
test	C	1324	26.8342
test	D	1224	24.8075
train	A	19146	21.7900
train	B	22726	25.8644
train	C	23891	27.1903
train	D	22103	25.1554

The most common training answer position is C at 27.19 percent. The distribution is close but not perfectly uniform, so answer-position bias should be monitored.

Text Length Findings

For readability, the original wide text-length table is split by text field. All values are word counts by split and school level.

Article length:

split	level	mean	median	range
dev	high	308.96	305.00	61-869
dev	middle	194.40	198.00	42-413
test	high	309.75	298.50	21-836
test	middle	197.25	195.00	20-422
train	high	313.27	307.00	3-1169
train	middle	193.69	190.00	2-529

Question length:

split	level	mean	median	range
dev	high	9.34	9.00	1-25
dev	middle	8.00	8.00	2-25
test	high	9.49	9.00	2-31
test	middle	7.99	8.00	0-28
train	high	9.51	9.00	0-61
train	middle	8.10	8.00	0-61

Correct-answer length:

split	level	mean	median	range
dev	high	6.45	6.00	0-37
dev	middle	4.75	4.00	1-33
test	high	6.58	6.00	1-25
test	middle	4.79	4.00	0-19
train	high	6.47	6.00	0-105
train	middle	4.68	4.00	0-28

Average option length:

split	level	mean	median	range
dev	high	6.23	6.25	0.00-22.25
dev	middle	4.50	4.00	0.75-17.25
test	high	6.34	6.50	1.00-20.25
test	middle	4.51	4.00	0.00-20.00
train	high	6.28	6.25	0.00-70.00
train	middle	4.44	4.00	0.00-23.25

Articles are much longer than questions and options. High-school passages are also longer than middle-school passages. This supports using separate article-question-option similarity features instead of relying only on raw combined text.

Questions Per Passage

split	level	count	mean	median	min	max
dev	high	1021	3.3800	3.0000	1	6
dev	middle	368	3.9022	4.0000	1	6
test	high	1045	3.3474	3.0000	1	6
test	middle	362	3.9669	4.0000	1	6
train	high	18726	3.3347	3.0000	1	6
train	middle	6409	3.9665	4.0000	1	7

Multiple questions share the same passage, so passage-aware checks are important when discussing leakage and dataset structure.

Leakage Check Findings

comparison	overlapping_example_ids	overlapping_exact_articles
train_vs_dev	0	0
train_vs_test	0	0
dev_vs_test	0	0

No exact example_id or article overlaps were found across train, dev, and test. This supports the leakage-free claim.

Option Length Bias

split	is_correct	count	mean	median	min	max
dev	False	14661	5.6456	5.0000	0	28
dev	True	4887	5.9515	6.0000	0	37
test	False	14802	5.7261	6.0000	0	28
test	True	4934	6.0614	6.0000	0	25
train	False	263598	5.6789	5.0000	0	91
train	True	87866	5.9533	6.0000	0	105

Correct options are only slightly longer on average than incorrect options. There is no strong length-only shortcut visible from this summary, but the feature remains worth monitoring.

Lexical Overlap Findings

split	is_correct	article_option_overlap_mean	article_option_overlap_median	question_option_overlap_mean	question_option_overlap_median
dev	False	0.0257	0.0229	0.0395	0.0000
dev	True	0.0285	0.0248	0.0372	0.0000
test	False	0.0259	0.0229	0.0400	0.0000
test	True	0.0288	0.0250	0.0388	0.0000
train	False	0.0255	0.0225	0.0398	0.0000
train	True	0.0283	0.0248	0.0391	0.0000

Correct options show slightly higher average article-option overlap than incorrect options. Question-option overlap is weaker and not consistently higher for correct answers. This supports the project feature engineering that uses article-option, question-option, and question-article cosine similarities, while also showing that lexical overlap alone is not enough.

Baseline Findings

split	baseline	BLEU	ROUGE-L	METEOR	exact_match_diagnostic
dev	Random option	0.3018	0.3969	0.3454	0.2570
dev	Majority option (C)	0.3254	0.4066	0.3569	0.2670
dev	Highest article-option overlap	0.3608	0.4488	0.4050	0.3192
dev	Highest question-option overlap	0.2727	0.3767	0.3239	0.2284
test	Random option	0.2934	0.3929	0.3389	0.2550

test	Majority option (C)	0.3230	0.4033	0.3526	0.2685
test	Highest article-option overlap	0.3733	0.4514	0.4065	0.3245
test	Highest question-option overlap	0.2856	0.3789	0.3249	0.2341

The article-overlap baseline is the strongest simple baseline, which means passage-option lexical matching is a useful signal. The trained model should aim to outperform these baselines on BLEU, ROUGE-L, and METEOR. Exact match is included only as a diagnostic, not as the requested final evaluation criterion.

Model A: Design, Training, Results

Model A covers answer verification and question generation. The final assignment-aligned verifier is a classical supervised LR + XGBoost ensemble trained on RACE train data with sparse text features, handcrafted similarity features, and within-question relative option features. The advanced verifier is a BERT multiple-choice backend. The generation side extracts answer spans, predicts question type, applies WH/cloze templates, and ranks generated candidates.

Model A Final Evaluation

Model A is the optimized answer-verification ensemble used in the notebook: Logistic Regression plus tuned XGBoost, blended with 0.2 Logistic Regression weight and 0.8 XGBoost weight. It was trained on the preprocessed RACE training split and evaluated on both dev and test splits.

split	BLEU	ROUGE-L	METEOR	exact_match_diagnostic
dev	0.3950	0.4892	0.4400	0.3653
test	0.4106	0.4922	0.4438	0.3739

Model A outperforms the strongest simple baseline, highest article-option overlap, on all three required metrics. On the test split, the strongest baseline scored BLEU 0.3733, ROUGE-L 0.4514, and METEOR 0.4065, while optimized Model A scored BLEU 0.4106, ROUGE-L 0.4922, and METEOR 0.4438. Exact match is included only as a diagnostic because the official requested evaluation criteria are BLEU, ROUGE-L, and METEOR.

Model A Algorithm Selection

After re-checking the assignment PDF, the safest required Model A baseline is a traditional supervised answer-verification pipeline. The rubric specifically rewards at least two traditional ML models, feature engineering, and a comparison table. Therefore, LR + XGBoost should be presented as the

assignment-aligned classical Model A baseline, while BERT multiple-choice should be presented as the advanced optional backend and best-performing measured Model A.

The first supervised comparison showed Logistic Regression and Linear SVM were stronger than the original XGBoost setup. After optimization, the final feature pipeline was improved without densifying the sparse text matrix. The optimized feature matrix keeps CountVectorizer/One-Hot features sparse and appends a compact dense handcrafted block containing:

- TF-IDF cosine similarity: question-option, article-option, and question-article.
- Lexical overlap: article-option Jaccard and question-option Jaccard.
- Length features: log option length, log question length, log article length, option/question length ratio, and option/article length ratio.
- Option-position one-hot features: A, B, C, and D.
- Question-relative features: each option's cosine, overlap, and length are compared against the other three options for the same question using rank, max flag, difference from group mean, difference from mean of other options, difference from max/min, and z-score.

XGBoost was then tuned for sparse text data using shallower regularized trees:

```
n_estimators=1000
learning_rate=0.03
max_depth=4
min_child_weight=4
subsample=0.85
colsample_bytree=0.65
reg_alpha=0.1
reg_lambda=2.0
tree_method='hist'
```

The optimized supervised comparison on the same preprocessing pipeline produced:

model	split	BLEU	ROUGE-L	METEOR	exact_match_diagnostic
Logistic Regression	dev	0.3878	0.4765	0.4273	0.3493
Linear SVM	dev	0.3842	0.4745	0.4249	0.3470
XGBoost	dev	0.3938	0.4789	0.4284	0.3526
LR + XGBoost Ensemble	dev	0.3909	0.4780	0.4290	0.3526
Logistic Regression	test	0.3963	0.4717	0.4214	0.3490
Linear SVM	test	0.4032	0.4768	0.4265	0.3539
XGBoost	test	0.4017	0.4828	0.4309	0.3608
LR + XGBoost Ensemble	test	0.4069	0.4847	0.4336	0.3640

Recommended classical choice: use the optimized LR + XGBoost ensemble as the final classical Model A baseline because it gives the best classical test BLEU, ROUGE-L, METEOR, and exact-match

diagnostic after feature optimization. Include Logistic Regression and Linear SVM as the two required traditional supervised baselines, and include XGBoost as the boosted tree comparison model.

Best feature strategy: keep One-Hot/CountVectorizer features as the assignment-aligned representation, keep the full matrix sparse, and append only a small dense block of engineered features. Do not convert the full text matrix to dense, because that would require tens of GB of memory. TF-IDF should be fit only on the training split and reused with transform() on dev/test to avoid leakage.

Question-Relative Feature Retraining

The classical verifier originally scored each (article, question, option) row mostly independently. This made the four option scores close because A/B/C/D share the same article and question context. To improve actual prediction capability, the dense feature block was extended with question-relative features. These features do not change the UI; they retrain the model so it can compare each option against the other options from the same question.

Added relative features:

- Question-option cosine rank within A/B/C/D.
- Article-option cosine rank within A/B/C/D.
- Article-option and question-option overlap ranks.
- Difference from the mean of the other options.
- Difference from the group mean, max, and min.
- Indicator for whether an option has the highest value for that feature.
- Within-question z-score for cosine, overlap, and option length.

Before/after retraining results:

model	split	BLEU	ROUGE-L	METEOR	exact_match_diagnostic
LR + XGBoost before relative features	dev	0.3909	0.4780	0.4290	0.3526
LR + XGBoost before relative features	test	0.4069	0.4847	0.4336	0.3640
LR + XGBoost with question- relative features	dev	0.3950	0.4892	0.4400	0.3653
LR + XGBoost with question- relative features	test	0.4106	0.4922	0.4438	0.3739

Finding: the relative-feature retrain improved the actual answer-selection model on all requested test metrics and improved exact-match diagnostic from 0.3640 to 0.3739. This confirms the earlier diagnosis: the classical model needed stronger within-question comparison signals, not only different score display.

Dual Correct/Wrong Ranker Experiment

A further experiment tested two-sided answer ranking. One ranker predicts which option is likely correct, while a second ranker predicts which option is likely wrong. The final resolver rewards options that are high in the correct-answer ranking and low in the wrong-answer ranking.

Application:

- Correctness ranker: the existing relative-feature LR + XGBoost model.
- Wrongness ranker: a second LR + XGBoost model trained with inverted labels, where incorrect options are positive examples.
- Resolver: combines normalized correct score, wrong score penalty, correct-rank bonus, low-wrong-rank bonus, and an agreement bonus for options that are top candidates in the correct ranking and bottom candidates in the wrong ranking.
- Tuned config: wrong penalty 1.25, rank weight 0.15, agreement bonus 0.10, conflict penalty 0.00, candidate K 2.

Results:

model	split	BLEU	ROUGE-L	METEOR	exact_match_diagnostic
Relative-feature correct ranker	dev	0.3950	0.4892	0.4400	0.3653
Relative-feature correct ranker	test	0.4106	0.4922	0.4438	0.3739
Dual correct/wrong ranker	dev	0.3958	0.4919	0.4427	0.3685
Dual correct/wrong ranker	test	0.4098	0.4905	0.4426	0.3723

Finding: the dual ranker improved dev performance, which means wrong-answer elimination is a useful signal. However, it did not outperform the current relative-feature LR + XGBoost model on the held-out test split. Therefore, it is kept as an experimental option in `src/model_a_inference.py` with `--use-dual-ranker`, but it is not selected as the final default classical Model A.

Blend Weight Tuning and SVM Ensemble Trial

After optimizing the feature set, blend-weight tuning was tested on the dev split. The tuning tried:

- LR + XGBoost raw probability blending.
- LR + XGBoost + Linear SVM blending, using sigmoid-transformed SVM decision scores.
- LR + XGBoost + Linear SVM blending with question-level score normalization.

The best weights from each tuning strategy were selected using dev exact-match diagnostic, then evaluated on test:

candidate	split	LR weight	XGB weight	SVM weight	BLEU	ROUGE-L	METEOR	exact_match_diagnostic
LR + XGBoost raw	dev	0.55	0.45	-	0.3976	0.4849	0.4354	0.3601
LR + XGBoost raw	test	0.55	0.45	-	0.4025	0.4783	0.4280	0.3565
LR + XGBoost + SVM raw	dev	0.20	0.35	0.45	0.4003	0.4868	0.4375	0.3634
LR + XGBoost + SVM raw	test	0.20	0.35	0.45	0.4035	0.4790	0.4288	0.3577
LR + XGBoost + SVM normalized	dev	0.25	0.50	0.25	0.3968	0.4849	0.4355	0.3616
LR + XGBoost + SVM normalized	test	0.25	0.50	0.25	0.4052	0.4809	0.4305	0.3591

Finding: adding SVM improved dev performance, but it did not outperform the optimized LR + XGBoost direction on the held-out test split. After the later question-relative feature retrain, the LR + XGBoost ensemble with 0.2 LR and 0.8 XGBoost remains the final classical Model A baseline because it gives the best classical test BLEU 0.4106, ROUGE-L 0.4922, METEOR 0.4438, and exact-match diagnostic 0.3739. The SVM-inclusive ensemble is still useful as an experiment because it demonstrates proper ensemble exploration, but it is not selected as the final classical model.

Optional BERT Backend

An optional BERT backend was added for Model A so the system can switch between the classical ML model and a neural answer verifier. The classical LR + XGBoost model remains important because it is aligned with the assignment rubric and gives a strong supervised baseline. BERT was then tested as an advanced neural extension because it can model the passage, question, and answer option jointly instead of relying only on sparse lexical features.

Files added for backend selection:

- `src/model_a_inference.py`: unified inference API with `--backend classical`, `--backend bert`, or `--backend auto`.
- `src/train_bert_model_a.py`: optional BERT fine-tuning script for answer verification.
- `src/train_bert_multiple_choice_model_a.py`: BERT multiple-choice fine-tuning script.
- `src/evaluate_bert_model_a.py`: BERT binary-verifier BLEU, ROUGE-L, and METEOR evaluator.
- `src/evaluate_bert_multiple_choice_model_a.py`: BERT multiple-choice BLEU, ROUGE-L, and METEOR evaluator.
- `src/model_a_train.py`, `src/inference.py`, and `src/evaluate.py`: rubric-facing entrypoints for training, inference, and evaluation.
- `ui/app.py`: Streamlit interface with a backend selector.
- `bts/requirements-bert.txt`: legacy optional BERT dependency list.
- `bts/results/bert_model_a_comparison.csv`: comparison of classical and BERT runs.

Classical inference example:

```
python src/model_a_inference.py \  
  --backend classical \  
  --article "Tom bought three red apples." \  
  --question "What did Tom buy?" \  
  --options '{"A":"three red apples","B":"a car","C":"a book","D":"a pen"}'
```

BERT multiple-choice training example:

```
pip install -r bts/requirements-bert.txt  
python src/train_bert_multiple_choice_model_a.py \  
  --model-name models/model_a/neural/bert \  
  --output-dir models/model_a/neural/bert_mc_30000q \  
  --epochs 1 \  
  --batch-size 4 \  
  --gradient-accumulation-steps 2 \  
  --max-length 256 \  
  --max-train-questions 30000 \  
  --max-dev-questions 3000 \  
  --learning-rate 1.5e-5 \  
  --eval-steps 750 \  
  --logging-steps 100 \  
  --early-stopping-patience 2 \  
  --label-smoothing 0.05
```

Continuation training on a fresh 30,000-question sample:

```
python src/train_bert_multiple_choice_model_a.py \
  --model-name models/model_a/neural/bert_mc_30000q \
  --output-dir models/model_a/neural/bert_mc_30000q_plus_new30000q \
  --epochs 1 \
  --batch-size 4 \
  --gradient-accumulation-steps 2 \
  --max-length 256 \
  --max-train-questions 30000 \
  --exclude-train-sample-size 30000 \
  --exclude-train-random-state 42 \
  --random-state 43 \
  --max-dev-questions 3000 \
  --learning-rate 1.0e-5 \
  --eval-steps 750 \
  --logging-steps 100 \
  --early-stopping-patience 2 \
  --label-smoothing 0.05
```

Current setup status: the optional dependencies have been installed in the project virtual environment, and a local pretrained BERT checkpoint has been saved at models/model_a/neural/bert. The environment was updated from an incompatible CUDA 13 PyTorch wheel to torch 2.6.0+cu124, matching the available driver support. PyTorch detects the RTX 3060 Laptop GPU with CUDA available. The strongest fine-tuned BERT checkpoint is models/model_a/neural/bert_mc_30000q_plus_new30000q.

GPU application, need, and finding:

- Application: BERT inference now uses --device auto, which selects CUDA when available. BERT training logs the active device and enables FP16 mixed precision by default on CUDA.
- Need: BERT is much heavier than the classical TF-IDF + supervised ML pipeline. GPU execution is needed to fine-tune within a realistic university-project time window and to avoid slow CPU-only transformer training.
- Finding: the machine has an RTX 3060 Laptop GPU with CUDA 12.4 driver support, and PyTorch verifies cuda_available=True. Because the GPU has 6 GB VRAM, the recommended BERT setting is batch size 4 with gradient accumulation 2, giving an effective batch size of 8 while reducing memory pressure.

BERT inference example after training:

```
python src/model_a_inference.py \
  --backend bert \
  --bert-checkpoint models/model_a/neural/bert_mc_30000q_plus_new30000q \
  --device auto \
  --article "..." \
  --question "..." \
  --options '{"A": "...", "B": "...", "C": "...", "D": "..."}'
```

The auto backend attempts to load BERT first and falls back to the classical model if BERT dependencies or checkpoint files are unavailable. This keeps the application robust during demonstration.

BERT experiment results:

model	objective	train questions	split	BLEU	ROUGE-L	METEOR	exact_match_diagnostic
LR + XGBoost	classical ensemble + relative features	87,866	dev	0.3950	0.4892	0.4400	0.3653
LR + XGBoost	classical ensemble + relative features	87,866	test	0.4106	0.4922	0.4438	0.3739
BERT	binary answer verifier	3,000	dev	0.3498	0.4391	0.3875	0.3098
BERT	binary answer verifier	3,000	test	0.3453	0.4259	0.3744	0.3030
BERT	multiple choice	3,000	dev	0.3975	0.4598	0.4119	0.3393
BERT	multiple choice	3,000	test	0.3820	0.4498	0.4007	0.3255
BERT	multiple choice	10,000	dev	0.4734	0.5460	0.4956	0.4367
BERT	multiple choice	10,000	test	0.4661	0.5315	0.4795	0.4258
BERT	multiple choice	30,000	dev	0.5085	0.5703	0.5203	0.4692
BERT	multiple choice	30,000	test	0.4965	0.5543	0.5043	0.4550
BERT	multiple choice	60,000 unique	dev	0.5547	0.6170	0.5653	0.5257
BERT	multiple choice	60,000 unique	test	0.5431	0.6073	0.5526	0.5172

Finding: the binary BERT verifier underperformed because it saw three negative option rows for every positive option, so option-row accuracy was not the same as selecting the correct answer. The BERT multiple-choice formulation was better because it optimized one four-way decision per question. Expanding from 10,000 to 30,000 training questions improved the full test results from BLEU 0.4661, ROUGE-L 0.5315, METEOR 0.4795, and exact-match 0.4258 to BLEU 0.4965, ROUGE-L 0.5543, METEOR 0.5043, and exact-match 0.4550. Continuing from the 30,000-question checkpoint on a fresh, non-overlapping 30,000-question sample improved the full test results again to BLEU 0.5431, ROUGE-L 0.6073, METEOR 0.5526, and exact-match 0.5172. The continuation run used a lower learning rate, small GPU batch size, gradient accumulation, label smoothing, validation every 750 steps, and early

stopping patience of 2. The final recommendation is to present LR + XGBoost as the required classical supervised baseline and BERT multiple-choice trained on 60,000 unique questions as the best-performing advanced Model A.

Model A Supervised Question Generation Upgrade

The generation side of Model A was upgraded from mostly fill-in-the-blank generation to a supervised hybrid generator. This is intentionally kept as a traditional ML plus template pipeline rather than a neural seq2seq generator, because it matches the assignment requirement for candidate sentence extraction, WH-word templates, and traditional machine-learning ranking.

Application in the project:

- Candidate extraction: each article is split into source sentences, then answer spans are extracted from named entities, numbers, dates/times, noun-like spans, and high-value content words.
- Question-type supervision: the original RACE training questions are converted into labels using their question patterns. The labels are cloze, who, what, where, when, why, how_many, which, and other.
- Classifier features: question text pattern, answer type, answer length, sentence length, answer position, TF-IDF/cosine similarity, lexical overlap, and option/answer features.
- Model comparison: Logistic Regression and Linear SVM are trained and compared as traditional supervised models. Linear SVM is selected as the saved question-type classifier.
- Typed templates: the predicted label controls the generated template, for example Who ...?, What ...?, Where ...?, When ...?, How many ...?, Which ...?, or the cloze fallback According to the passage, ... _
- Candidate ranking: a Random Forest ranker scores each generated Q/A candidate using classifier confidence, answer type, overlap features, TF-IDF article/question similarity, generated question length, answer length, and template type. The UI display score then blends this ranker score with the extraction heuristic and a typed-question bonus so rare WH types remain visible.
- Streamlit application: the UI now supports ML typed generation, Cloze fallback, and Both, and displays each generated candidate with question type, classifier confidence, ranker score, answer, generated question, and source sentence.

Need for this upgrade:

- The older generator was reliable but narrow because most outputs were fill-in-the-blank questions.
- RACE contains many WH questions, so a stronger Model A generator should produce different question types rather than only blanking words.
- A supervised classifier makes the choice of question type data-driven, while templates keep the generated questions explainable and assignment-aligned.
- The ranker prevents the UI from showing only the first extracted span; it promotes candidates that better match the article, answer, and generated question.

Training split isolation:

- The classifier and ranker are trained only from the RACE train split.
- Dev and test are used only for evaluation.
- TF-IDF/vectorizer objects are fit on training data and reused with transform() for dev/test, avoiding leakage.

Question-type label distribution:

split	cloze	what	which	other	why	how_m any	who	where	when
train	31471	11163	7289	4202	2704	1123	838	716	467
dev	2576	956	547	346	217	76	64	60	45
test	2645	903	610	344	197	81	62	57	35

Finding: the dataset is heavily imbalanced. Cloze-like and what questions dominate, while who, where, and when are much smaller classes. This explains why classifier accuracy is moderate and macro-F1 is low: macro-F1 penalizes poor performance on rare classes. The final runtime generator therefore combines the classifier with answer-type safeguards, so dates can still produce when, locations can produce where, numbers can produce how_many, and named entities can produce who when appropriate.

Question-type classifier results:

model	split	accuracy	macro-F1
Logistic Regression	dev	0.3382	0.1641
Logistic Regression	test	0.3405	0.1593
Linear SVM	dev	0.4571	0.1838
Linear SVM	test	0.4710	0.1833

Finding: Linear SVM is the better traditional classifier, so it is saved as the active question-type classifier under models/model_a/traditional/generator/generator_artifacts.joblib. The low macro-F1 is expected from the class imbalance, but the classifier still gives useful type probabilities and separates broad question families better than the logistic baseline.

Generation ranker results:

split	top-1 candidate accuracy	mean reciprocal rank	evaluated questions
dev	0.9792	0.9850	1200
test	0.9833	0.9870	1200

Finding: the ranker is strong at selecting a candidate span that matches the original RACE answer/sentence setting. This is useful for the Streamlit demo because the generated list is ordered by likely usefulness instead of by raw extraction order.

Template behavior examples:

type	example output style
cloze	According to the passage, Tom bought _ red apples.
who	Who bought three red apples from the market?
what	What is Wang Lin?
where	Tom bought three red apples where?
when	He shared them with his sister when?
why	Why is this statement true according to the passage: ...?
how_many	Tom bought how many red apples from the market?
which	Which answer ...?

The WH templates are intentionally simple and explainable. If the generated WH question is weak or the classifier/ranker artifacts are unavailable, the generator falls back to cloze generation. This keeps the system robust during marking and demonstration.

Generator artifacts:

- models/model_a/traditional/generator/generator_artifacts.joblib
- models/model_a/traditional/generator/question_type_classifier_results.csv
- models/model_a/traditional/generator/generation_ranker_results.csv
- models/model_a/traditional/generator/question_type_label_distribution.csv
- models/model_a/traditional/generator/generation_examples.csv

Conclusion for Model A generation: the upgraded generator satisfies the assignment's Model A generation requirement because it performs candidate sentence extraction, applies WH-word and cloze templates, uses supervised traditional ML to choose question type, and uses a learned ranker to order generated candidates. BERT remains the advanced verifier backend, not the generator.

Model A Unsupervised and Semi-Supervised Clustering Layer

Section 4.2.2 of the assignment requires at least one unsupervised or semi-supervised approach for Model A. This project adds K-Means clustering and dendrogram analysis as an auxiliary Model A layer. It is not used to replace the supervised verifier, because the dataset has labelled answers and the supervised models are stronger. Instead, clustering is used for pattern discovery, generation metadata, and an optional experimental cluster-prior score.

Application in the project:

- Question-form clustering: original RACE questions are represented with TF-IDF features over question text plus answer metadata. K-Means is then used to group questions without using the question-type label during fitting.

- Dendrogram analysis: hierarchical clustering dendrograms are produced on sampled reduced vectors to inspect whether natural groupings exist before selecting K.
- Best-K selection: K is selected by comparing silhouette scores across candidate values.
- Dimensionality reduction: TruncatedSVD is applied before clustering so K-Means can work on compact dense vectors rather than the full sparse text space.
- Answer-option clustering: each (article, question, option) row is clustered using TF-IDF, cosine similarity, lexical overlap, length, and option-position features.
- Runtime use: generated Q/A candidates now show the unsupervised question cluster ID, dominant cluster type, cluster purity, and cluster distance in Streamlit. Classical inference can optionally apply a small cluster-prior blend with --unsupervised-weight, but the default remains the supervised LR + XGBoost score.

Question-type K-Means results:

split	best K	silhouette	question-type purity
train	4	0.1898	0.6247
dev	4	0.2042	0.6035
test	4	0.1853	0.6490

Question-cluster findings:

- Cluster 2 is a strong which/which of the following cluster, with purity 0.8824.
- Cluster 3 captures many title-style what questions, although it overlaps with which and cloze-like questions.
- Clusters 0 and 1 are broad cloze/factual clusters, which reflects the heavy cloze and factual-question bias in RACE.
- The silhouette scores are positive but modest, so question types are not perfectly separated by unsupervised TF-IDF alone.

Answer-option K-Means results:

split	best K	silhouette	correctness purity	cluster-prior answer accuracy
train	5	0.3179	0.7500	-
dev	5	0.3171	0.7500	0.2700
test	5	0.3183	0.7500	0.2675

Answer-cluster findings:

- The answer-option clusters are geometrically clearer than the question-form clusters, shown by silhouette around 0.318.
- Correctness purity is 0.75 because every RACE question has one correct option and three incorrect options, so a cluster can appear pure by mostly containing incorrect options.

- The cluster-prior answer accuracy is only 0.2700 on dev and 0.2675 on test, close to simple random/majority behavior.
- Therefore, K-Means is useful for unsupervised analysis and auxiliary metadata, but it should not replace the supervised LR + XGBoost or BERT verifiers.

Artifacts:

- models/model_a/traditional/unsupervised/model_a_unsupervised.joblib
- models/model_a/traditional/unsupervised/question_cluster_dendrogram.png
- models/model_a/traditional/unsupervised/answer_cluster_dendrogram.png
- models/model_a/traditional/unsupervised/question_cluster_eval.csv
- models/model_a/traditional/unsupervised/answer_cluster_eval.csv
- models/model_a/traditional/unsupervised/question_cluster_profiles.csv
- models/model_a/traditional/unsupervised/answer_cluster_profiles.csv

Conclusion for unsupervised Model A: K-Means satisfies the assignment's unsupervised learning requirement and provides useful cluster-level findings. The best role for it is EDA, generation explanation, and optional auxiliary scoring. The final answer-verification decision remains supervised because labelled RACE answers make supervised learning more appropriate and empirically stronger.

Model B: Design, Training, Results

Model B covers distractor and hint generation. It uses the passage, question, and correct answer to generate three plausible distractors and three progressive hints. The design combines passage candidate extraction, question-type and answer-type control, TF-IDF/overlap/length/frequency features, supervised distractor ranking, diversity filtering, and evidence-sentence ranking.

Model B Distractor and Hint Generator

Model B has now been implemented as a supervised hybrid distractor and hint generator. Its input is the passage, question, and correct answer. Its output is three ranked distractors and three graduated hints.

Need in the project:

- The assignment separates answer verification/question generation from distractor and hint generation. Model B fills that second role.
- RACE already provides official wrong options, so these can be used as supervised positive examples for plausible distractors.
- The question-type labels created for Model A are useful here because distractors should match the expected answer type. A how_many question needs numeric alternatives, a when question needs time/date alternatives, a who question needs person/name alternatives, and cloze/what questions need phrase-level alternatives.
- Hints should guide the student toward the right sentence without directly exposing the answer, so they need sentence ranking and answer masking.

Application in the project:

- Candidate extraction: Model B extracts passage candidates using the same span extraction tools used by Model A generation, plus additional passage-grounded action phrase and prepositional phrase patterns. During supervised training/evaluation it also includes the four RACE answer options so official wrong answers can be labelled as positive distractors and the correct option can be labelled as a negative candidate.
- Question-type control: `infer_question_type()` maps each question to cloze, who, what, where, when, why, how_many, which, or other. This type is used as a categorical ML feature and as a compatibility rule for candidate answer types.
- Distractor features: TF-IDF text features over question/correct answer/candidate/source sentence, answer type, question type, candidate source, word/character lengths, length ratios, lexical overlap, one-hot/binary cosine similarity to the correct answer, character-level similarity, passage frequency, and sentence position.
- Distractor ranker: Logistic Regression and Random Forest were compared. The selected runtime ranker is Logistic Regression with full text + structured features. A diversity filter removes near-duplicates and subset/superset candidates.
- Hint features: sentence/question overlap, sentence/answer overlap, sentence length, sentence position, whether the sentence contains the answer, and whether it contains important question terms.
- Hint ranker: Logistic Regression ranks passage sentences. The runtime now turns the ranked evidence into three non-repeating levels: Hint 1 is vague, Hint 2 is moderate/contextual, and Hint 3 is the near-explicit masked sentence.
- Runtime passage-only generation: the Streamlit demo defaults to distractors from the passage rather than the official RACE option pool. Because the supervised distractor labels come mainly from official RACE options, passage-only generation relies more heavily on heuristic type, answer-form, overlap, frequency, and diversity rules.

Training configuration:

```
train questions: 30,000
dev/test evaluation questions: 2,500 each
negative passage candidates per question: 8
Random Forest max training candidate rows: 90,000
```

Distractor ranker results:

model	split	Precision @3	Recall@3	F1@3	candidate precision	candidate recall	candidate F1
Logistic Regression	dev	0.9995	1.0000	0.9997	0.9995	1.0000	0.9997
Logistic Regression	test	0.9999	1.0000	0.9999	0.9999	1.0000	0.9999

Random Forest	dev	0.9995	1.0000	0.9997	0.9995	1.0000	0.9997
Random Forest	test	0.9999	1.0000	0.9999	0.9999	1.0000	0.9999

Distractor generation text-overlap metrics:

split	mode	BLEU	ROUGE-L	METEOR	exact text match
dev	option pool	0.8688	0.9672	0.8477	0.9647
test	option pool	0.8998	0.9770	0.8729	0.9760
dev	passage grounded	0.0142	0.0955	0.0674	0.0195
test	passage grounded	0.0149	0.0737	0.0537	0.0081

These BLEU/ROUGE-L/METEOR scores are computed by `src/evaluate_model_b_generation.py` over 500 dev and 500 test questions per mode. In option-pool mode, the candidate pool includes the official RACE answer options, so high text overlap means the ranker recovers the labelled wrong answers. In passage-grounded mode, Model B intentionally generates wrong options from passage spans rather than copying the official multiple-choice distractors, so lexical-overlap metrics are lower even when the distractors are passage-related and usable for the demo.

Important interpretation: these are option-pool ranking metrics. The supervised candidate pool contains the official RACE wrong options, the correct option, and sampled passage candidates. Therefore, the result shows that Model B can identify official plausible distractors from a labelled candidate pool. Runtime generation in the UI now defaults to passage-grounded candidates, so its output is more faithful to the assignment demo but naturally less perfect than option-pool evaluation.

Hint ranker results:

model	split	top-1 sentence accuracy	mean reciprocal rank	evaluated questions
Logistic Regression	dev	0.7232	0.8362	2500
Logistic Regression	test	0.7392	0.8436	2500

Findings:

- Official RACE distractors are usually easy to identify once included in the candidate pool because they are semantically plausible answer options but usually do not appear as exact supporting facts in the passage.

- The correct answer often appears in or near the evidence sentence, so features such as candidate-in-article, source sentence overlap, and answer/source relation help reject it as a distractor.
- Hint generation is harder than option-pool distractor ranking because the model must choose one evidence sentence from the full passage. Test top-1 accuracy of 0.7392 and MRR of 0.8436 show that the correct supporting sentence is usually ranked near the top.
- Question-type labels add practical control: numeric questions produce numeric distractors more often, entity questions favor named candidates, and hints can say whether the answer is a person, place, time/date, number, or general phrase.

Example generated outputs:

question type	correct answer	generated distractors	graduated hint behavior
cloze	communicate	passage phrases such as takes care; play different; meaning sitting	Hint 1 gives only passage region/type, Hint 2 gives sentence location and keywords, Hint 3 masks communicate
how_many	Three	passage numbers such as about 70; more than 80; five	Hint 1 says the answer is a number/quantity before the masked sentence is shown
cloze	make sure that nobody chats in class	takes care of the whole group; play different roles; facing each other	final hint masks overlapping answer words in the source sentence

UI alignment with assignment Section 6.2:

- Screen 1, Article Input: the app provides a passage text area, RACE Previous/Next/Random sample loading, a Clear Passage option for custom input, and a Submit button. Submit runs Model A question/correct-answer generation first, then Model B distractor and hint generation.
- Screen 2, Question & Answer Quiz View: the app displays the generated question, correct option mixed with three Model B distractors, a user answer selector, a Check button, and colour-coded correctness feedback. If the RACE passage was not cleared or edited, the exact RACE question, correct option, and official options can also be viewed or loaded.
- Screen 3, Hint Panel: hints are hidden initially and revealed one at a time. Hint 1 is vague, Hint 2 is more specific, Hint 3 is near-explicit, and Reveal Answer appears only after all hints have been used.
- Screen 4, Developer / Analytics Dashboard: the app reports Model A last-N accuracy/precision/recall/F1, a confusion matrix, inference latency, CSV session-log export, Model B last-N latency, and the saved Model B distractor/hint evaluation tables.

Artifacts:

- models/model_b/traditional/model_b_artifacts.joblib
- models/model_b/traditional/model_b_distractor_results.csv
- models/model_b/traditional/model_b_hint_results.csv
- models/model_b/traditional/model_b_generation_text_metrics.csv
- models/model_b/traditional/model_b_examples.csv
- models/model_b/traditional/model_b_training_summary.json

Conclusion for Model B: the implementation matches the assignment techniques by combining candidate extraction, TF-IDF features, one-hot/binary cosine similarity, character matching, passage frequency, Logistic Regression/Random Forest ranking, diversity filtering, BLEU/ROUGE-L/METEOR distractor evaluation, and rule-based plus ML-ranked hints. The Streamlit app now exposes Model B and can load generated distractors back into the Model A verifier for end-to-end testing.

User Interface Description

The Streamlit interface is organized as an end-to-end quiz lab. The user starts with a RACE passage or custom article, runs Model A question/correct-answer generation, receives Model B distractors and hints, selects an answer, and checks the result. The interface also supports backend switching between BERT multiple-choice and the classical LR + XGBoost verifier, manual A/B/C/D option entry, RACE sample browsing, generated-question details, option scores, confidence charts, latency tracking, session analytics, CSV export, and downloadable JSON outputs.

Evaluation & Discussion

The strongest simple baseline is highest article-option overlap, confirming that lexical matching is useful. The final classical Model A improves over that baseline with test BLEU 0.4106, ROUGE-L 0.4922, METEOR 0.4438, and exact-match diagnostic 0.3739. The BERT multiple-choice backend is the strongest measured verifier, reaching test BLEU 0.5431, ROUGE-L 0.6073, METEOR 0.5526, and exact 0.5172. Model A generation is strongest at candidate ranking, with test top-1 candidate accuracy 0.9833 and MRR 0.9870. The K-Means layer is useful for analysis and metadata but not strong enough to replace supervised answer verification. Model B achieves near-perfect option-pool distractor ranking and strong option-pool text overlap against official wrong answers, with test BLEU 0.8998, ROUGE-L 0.9770, and METEOR 0.8729. Those numbers should be interpreted carefully because the supervised pool includes official RACE wrong options; passage-grounded runtime generation is more faithful to the demo but naturally lower on lexical-overlap metrics. Hint ranking is more realistic and harder, with test top-1 sentence accuracy 0.7392 and MRR 0.8436.

Limitations & Future Work

The classical verifier still depends heavily on lexical and engineered similarity features, so it can miss answers requiring deeper paraphrase or multi-sentence reasoning. The BERT backend performs better but is more expensive to train and run, especially without GPU access. The question generator uses explainable templates rather than a neural sequence-to-sequence model, so some WH questions can sound rigid. The Model B distractor metrics are strongest in the option-pool setting; passage-only runtime distractors are more faithful to the demo but naturally less perfect. Future work should

improve semantic distractor quality, add stronger neural or retrieval-augmented generation for questions, evaluate with human judges, calibrate model confidence, and run ablation studies for feature groups and question types.

Generated EDA Artifacts

- `report/eda_outputs/answer_distribution.png`
- `report/eda_outputs/level_distribution.png`
- `report/eda_outputs/text_length_boxplots.png`
- `report/eda_outputs/option_length_bias.png`
- `report/eda_outputs/lexical_overlap.png`
- `report/eda_outputs/baseline_metrics.png`

Conclusion

The dataset is clean after preprocessing, the official splits show no exact leakage, and the EDA justifies the answer-verification pipeline. The added preprocessing gives the project a reproducible schema, validates data quality, and prepares answer text for BLEU, ROUGE-L, and METEOR evaluation. The final classical supervised baseline is LR + XGBoost with question-relative option features, while the strongest measured Model A is BERT multiple-choice trained on 60,000 unique questions with GPU acceleration. Model B now adds supervised distractor ranking and graduated hint generation, giving the project both the answer-verification/generation side and the distractor/hint side required by the assignment.

References

- [1] Guokun Lai, Qizhe Xie, Hanxiao Liu, Yiming Yang, and Eduard Hovy. 2017. RACE: Large-scale ReAding Comprehension Dataset From Examinations. EMNLP 2017. <https://aclanthology.org/D17-1082/> (<https://aclanthology.org/D17-1082/>)
- [2] D. R. Cox. 1958. The Regression Analysis of Binary Sequences. *Journal of the Royal Statistical Society: Series B*, 20(2), 215-232. <https://doi.org/10.1111/j.2517-6161.1958.tb00292.x> (<https://doi.org/10.1111/j.2517-6161.1958.tb00292.x>)
- [3] Leo Breiman. 2001. Random Forests. *Machine Learning*, 45, 5-32. <https://doi.org/10.1023/A:1010933404324> (<https://doi.org/10.1023/A:1010933404324>)
- [4] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. KDD 2016. <https://doi.org/10.1145/2939672.2939785> (<https://doi.org/10.1145/2939672.2939785>)
- [5] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL 2019. <https://aclanthology.org/N19-1423/> (<https://aclanthology.org/N19-1423/>)
- [6] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. BLEU: a Method for Automatic Evaluation of Machine Translation. ACL 2002. <https://aclanthology.org/P02-1040/> (<https://aclanthology.org/P02-1040/>)

[7] Chin-Yew Lin. 2004. ROUGE: A Package for Automatic Evaluation of Summaries. Text Summarization Branches Out. <https://aclanthology.org/W04-1013/> (<https://aclanthology.org/W04-1013/>)

[8] Satanjeev Banerjee and Alon Lavie. 2005. METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments. ACL Workshop on Intrinsic and Extrinsic Evaluation Measures. <https://aclanthology.org/W05-0909/> (<https://aclanthology.org/W05-0909/>)